

CHINTU — Interview Guide

Local-first AI assistant in Java · Spring Boot + LangChain4j + Ollama
Repo: github.com/pankajsharma21/local-ai-assistant

Contents

- 1. 30-second pitch
- 2. Architecture — ek engine, alag data
- 3. RAG kya hai (simple explanation)
- 4. Saare tools (8 tools)
- 5. Voice — speech in aur speech out
- 6. UI features
- 7. Real bugs jo mile aur fix kiye ← **sabse important**
- 8. Performance findings (real measurements)
- 9. Live demo flow
- 10. Anticipated interview questions

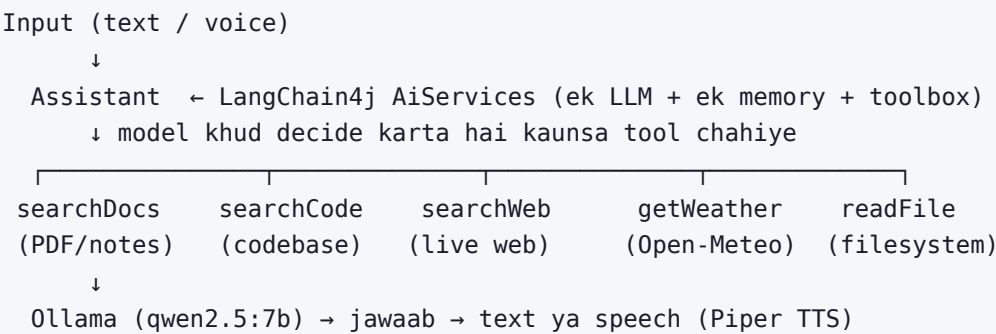
1. 30-second pitch

Aap bolo: "Maine CHINTU banaya hai — ek Java-based AI assistant jo **local-first** hai. Model mere apne laptop pe chalta hai Ollama ke through, koi OpenAI ya ChatGPT API nahi. Ye normal chat kar sakta hai, mere documents se sawaal-jawaab (RAG), mere codebase ke baare mein sawaal, current facts web se, weather, aur voice mein bol bhi sakta hai aur sun bhi sakta hai. Sab ek hi engine se — bas data aur tool alag hain."

Accuracy note: "fully offline" mat bolna. LLM, embeddings, vector store, voice — sab local hai, lekin Wikidata / Wikipedia / web search / weather internet call karte hain. Isliye **"local-first"** bolo, "fully offline" nahi. Interviewer isko pakad sakta hai.

2. Architecture — ek engine, alag data

Ye sabse important insight hai jo poore project ko explain karta hai:



Aap bolo: "Core mein ek hi LLM hai. Maine if-else likh ke route nahi kiya — LangChain4j ke @Tool annotation se maine tools register kiye, aur model khud padhta hai ki kaunsa tool kis sawaal ke liye chahiye. Doc assistant aur code assistant ka code almost identical hai — bas ek 'docs' vector store dekhta hai, doosra 'code'. Isliye ye 5 alag app nahi, ek engine hai."

Tech stack

Layer	Kya use kiya	Kyun
LLM	Ollama + qwen2.5:7b	Local, free, ek command se setup
Orchestration	LangChain4j	Java ka LangChain — tool calling, memory, RAG
Embeddings	all-MiniLM-L6-v2 (ONNX)	JVM ke andar chalta hai — koi server nahi
Vector store	InMemory + JSON file	Single user ke liye kaafi; Chroma/pgvector overkill
Speech-to-text	whisper.cpp (small, multilingual)	Local binary, Hindi/Hinglish support
Text-to-speech	Piper	Local binary, koi cloud TTS nahi
Web/API	Spring Boot 4	REST + static UI

3. RAG kya hai (simple explanation)

Aap bolo: "RAG = Retrieval-Augmented Generation. Closed-book exam vs open-book exam socho. Normal LLM closed-book hai — sirf training ka yaad hai, mere documents nahi jaanta. RAG usko open-book bana deta hai: sawaal aane pe main related paragraphs apne documents se dhoond ke LLM ko dikha deta hoon, phir wo unko padh ke jawaab deta hai."

Technical steps:

- 1. Document ko chhote chunks mein todo (500 chars, 50 overlap)
- 2. Har chunk ko embedding model se vector (numbers) mein badlo — meaning capture hota hai
- 3. Vectors JSON file mein store karo (mera chhota vector DB)
- 4. Sawaal aaye to usko bhi vector banao, cosine similarity se similar chunks dhoondo
- 5. Wo chunks LLM ko prompt mein do: "ye rahe excerpts, isi se jawaab do"

Agar poochhe "embedding bhi local hai?": Haan — all-MiniLM ONNX model seedha JVM ke andar chalta hai, Ollama ko bhi call nahi karta. Ek moving part kam.

4. Saare tools (8 tools)

Tool	Kaam	Setup chahiye?
<code>searchDocs</code>	User ke PDF/notes/markdown mein search	Nahi
<code>searchCode</code>	Ingested codebase mein search	Nahi
<code>listDocuments</code>	Kaunse documents ingested hain (+ latest ka content)	Nahi
<code>readFile</code> / <code>listFiles</code>	Filesystem read (sandboxed)	Nahi
<code>searchWikidata</code>	Structured facts — "latest version of X" ke liye best	Nahi, no key
<code>searchWikipedia</code>	Encyclopedic context	Nahi, no key
<code>searchWeb</code>	Live web (Marginalia); Tavily optional	Nahi, no key
<code>getWeather</code>	Current weather + forecast (Open-Meteo)	Nahi, no key

Fallback chain — code mein, prompt mein nahi

`searchWeb` → Tavily (agar key hai) → Marginalia → Wikidata → Wikipedia

Aap bolo: "Ye chain maine **code mein** likhi hai, prompt mein nahi. Kyunki maine test karke dekha — model ek tool call karta hai, uska 'not configured' message padhta hai, aur wahin ruk jaata hai. Dusra tool call kabhi nahi karta. Ye chhote models ki known limitation hai: **single-hop tool use**. Prompt mein likhne se fix nahi hua, isliye deterministic fallback code mein daali."

Security: do alag trust levels

Aap bolo: "`readFile` / `listFiles` — jo **LLM khud call karta hai** — project folder tak sandboxed hain, kyunki koi ingested document mein prompt-injection ho sakta hai. Lekin `/api/ingest/path` — jo **insaan khud** path type karke call karta hai — sandboxed nahi, kyunki wo same trust level hai jaise file text editor mein kholna. Iske badle server sirf `127.0.0.1` pe bind hota hai, poore network pe nahi."

5. Voice — speech in aur speech out

Aap bolo: "Voice koi alag AI pipeline nahi hai — sirf **input/output wrapper** hai. Mic se audio → whisper.cpp (local STT) → wahi text usi assistant ko → jawaab → Piper (local TTS) → speaker. Agent ka core logic bilkul nahi badalta."

- **Model:** whisper `small` multilingual — Hindi/Hinglish aur Indian accent handle karta hai
- **Language:** `auto` — whisper ka default `en` hai, isko explicitly set karna padta hai
- **UI:** har reply pe  button, aur sidebar mein "read replies aloud automatically" toggle

6. UI features

- **Recent chats** — localStorage mein save, click karke resume; server-side memory bhi continue hoti hai (sirf transcript replay nahi)
- **In-chat ingestion** —  button se menu: upload file / docs folder / code folder / koi bhi path

- **Live model switcher** — header dropdown, bina restart ke model badlo, chat memory bhi carry over hoti hai
- **Markdown rendering**, auto-grow composer, typing indicator, copy button, read-aloud
- **Document delete** — `DELETE /api/documents?name=...`

Aap bolo: "Model switching ke liye real refactor karna pada — LangChain4j ka AIServices build time pe ChatModel bake kar deta hai, koi `setModel()` nahi hai. Isliye `AssistantService` proxy ko rebuild karta hai, lekin chat memory apne khud ke map mein rakhta hai jo har rebuild ko same lambda se milti hai — isliye model badalne pe history nahi jaati."

7. Real bugs jo mile aur fix kiye

Ye section sabse zyada value deta hai interview mein — dikhata hai ki tumne sirf tutorial follow nahi kiya, actually debug kiya. Har bug ka **symptom** → **root cause** → **fix** yaad rakho.

Bug 1 — Code ingestion 0 files (apne hi naam ki wajah se)

Symptom: Code ingest karne pe 0 files. **Root cause:** Ignore-folder check poore *absolute path* pe chal raha tha, aur default folder ka naam hi `data/code` tha — `data` `IGNORED_DIRS` mein tha, isliye har file skip. **Fix:** path ko root ke relative karke check kiya.

Bug 2 — Spring Boot 4 vs LangChain4j: do alag Jackson

Symptom: Type definition error: `JsonNode`. **Root cause:** Spring Boot 4 naya **Jackson 3** (`tools.jackson`) use karta hai, LangChain4j purana **Jackson 2** (`com.fasterxml`). RestClient ka auto-converter Jackson-2 ka `JsonNode` samajh hi nahi paata. **Fix:** response ko raw String lo, apne `ObjectMapper` se parse karo.

Bug 3 — Wikipedia ka galat endpoint

Symptom: "latest Java version" pe kuch nahi milta. **Root cause:** `opensearch` endpoint sirf *title prefix* match karta hai — ye query zero results deti hai. **Fix:** `action=query&list=search` (real full-text search) use kiya.

Bug 4 — Spring UriBuilder ne SPARQL kha liya

Symptom: Wikidata tool silently fail. **Root cause:** `UriBuilder {...}` ko URI template placeholder samajhta hai, aur SPARQL braces se bhara hota hai → "Not enough variable values available to expand". **Fix:** pre-encoded `java.net.URI` khud banaya.

Bug 5 — Voice: browser WAV record kar hi nahi sakta

Symptom: Har recording pe `failed to read audio file`. **Root cause:** `MediaRecorder` sirf **webm/opus** deta hai, WAV nahi — aur `whisper.cpp` webm nahi padh sakta. **Fix:** `ffmpeg` install nahi tha (aur root access bhi nahi), isliye browser mein hi Web Audio API se decode karke, 16kHz mono resample karke, khud WAV bytes encode kiye.

Bug 6 — Intel iGPU chup-chaap output corrupt kar raha tha

Symptom: "What is 2+2?" ka jawaab `[]([]([]([]... infinite`. **Root cause:** Maine speed test ke liye `OLLAMA_IGPU_ENABLE=1` kiya tha. `ollama ps` "100% GPU" dikha raha tha, lekin Vulkan path generation corrupt kar raha tha. **Fix:** CPU-only par wapas. **Lesson:** maine sirf *latency* measure ki thi, *output quality* nahi — speed benchmark is failure ko kabhi nahi pakadta.

Bug 7 — min-score chup-chaap valid matches drop kar raha tha

Symptom: "documents mein hai lekin bolta hai nahi mila". **Root cause:** `min-score 0.6`, lekin all-MiniLM genuinely relevant chunks ko 0.4–0.6 deta hai. Valid matches silently filter ho rahe the. **Fix:** 0.35 kiya, aur ab tool *log karta hai* ki best rejected score kya tha — taaki agli baar ye bug dikhe, chhupe nahi.

Bug 8 — Model raw JSON leak kar raha tha

Symptom: "hi" bolne pe jawaab aaya `{"name": "listDocuments", "parameters": {}}`. **Root cause:** llama3.2 tool-call blob ko *text ki tarah print* kar raha tha — real tool call nahi tha. Prompt mein mana karne se bhi nahi ruka (3 mein se 3 baar hua). **Fix:** code mein filter + ek retry. Aur retry karte waqt failed exchange ko memory se hata bhi diya, warna model apni hi galti pe maafi maangne lagta tha ("It seems like I made a mistake") — jo internal machinery user ko dikha deta hai.

8. Performance findings (real measurements)

Ye numbers actual measure kiye hain — interview mein "maine measure kiya" bolna bahut strong hai.

Task (warm model)	llama3.2 (3B)	qwen2.5:7b
Greeting	2+ min, raw JSON leak	3.8s , clean
Follow-up (memory)	galat, rambling	7.6s , correct
"Write Java code to add 2 numbers"	2m 02s	35s

Aap bolo: "Sabse counter-intuitive finding: **chhota model actually slow nikla**. Raw token rate mein 3B tez hai (7.5 vs 3.7 tok/s), lekin wo bekaar ke tool calls pe **extra round-trips** kharch karta hai — Ollama ke logs mein maine dekha ek sawaal pe do model calls ho rahe the. Isliye asli metric tokens-per-second nahi, **round-trips per answer** hai."

Aur kya seekha

- **Cold start** sabse bada cost tha — Ollama 5 min baad model unload kar deta hai, phir har sawaal 1–2 min reload pe kharch karta tha. `OLLAMA_KEEP_ALIVE` se fix.
- Lekin sirf keep-alive lagane se **naya problem** aaya — dono models RAM mein pin ho gaye (7.7GB), machine choke ho gayi. `OLLAMA_MAX_LOADED_MODELS=1` bhi lagana zaroori tha.
- **Prompt overhead:** system prompt + 8 tool descriptions = **2066 tokens**, yaani 4096 context ka aadha hissa har request pe. Trim karke ~1000 kiya.
- **RAM ≠ speed:** 2.4GB free karne se sirf 8% farak pada. RAM *model load* ko affect karti hai, *generation speed* ko nahi — wo CPU-bound hai.

9. Live demo flow

1. `GET /api/health` — Ollama reachable, model, voice, web search sab green
2. 📎 se koi document upload karo → "indexed" confirmation
3. Us document ke baare mein poochho → exact line milti hai (proof: guess nahi, retrieve kar raha hai)
4. "What is the latest Java LTS version?" → Wikidata se live data, purani memory se nahi
5. "today weather in gurugram" → real current temperature
6. Header dropdown se model switch karo → bina restart ke, chat history bani rehti hai
7. Kisi reply pe 🔊 click karo → wo bol ke sunata hai
8. Sidebar mein "New conversation", phir purani chat pe wapas click karo → resume ho jaati hai

Demo tip: CPU-only pe har jawaab 30–100 second leta hai. Interview se pehle app start karke ek dummy sawaal pooch lo taaki model RAM mein warm ho jaaye — warna pehla jawaab 2 minute lega aur awkward lagega.

10. Anticipated interview questions

Sawaal	Jawaab
Ye fully offline hai?	LLM, embeddings, vector store, voice — sab local. Lekin Wikidata/Wikipedia/web/weather internet call karte hain. Isliye local-first bolo. Ye conscious trade-off tha: accuracy ke liye controlled exception.
Cloud API kyun nahi?	Privacy (confidential docs/code kisi server pe nahi jaate) aur cost (per-request charge nahi).
Tool calling kaise kaam karta hai?	LLM ko tool descriptions dikhte hain, wo khud decide karta hai. Main if-else route nahi karta.
Model kaise badloge?	Header dropdown se live, ya <code>application.yml</code> mein ek line. Restart nahi chahiye.
Production mein scale kaise?	Vector store ko pgvector/Chroma mein swap, GPU wali machine, aur per-user session isolation.
Multiple users?	Haan — har session ka apna <code>sessionId</code> aur memory hai.
Sabse bada challenge?	Bug 6 (iGPU corruption) ya Bug 8 (JSON leak) sunao — dono mein root cause non-obvious tha aur prompt se fix nahi hua, code se hua.
Agla step kya?	Agentic: write-capable tools (sandboxed workspace), multi-step planning loop, long-term memory. Abhi sab tools read-only hain.
Testing kaise ki?	Har feature app ke through end-to-end verify kiya — logs se confirm kiya ki sahi tool fire hua, sirf output dekh ke maan nahi liya.

Bottom line — ek line yaad rakho

Aap bolo: "Ek model, ek agent, aath tools, alag-alag data — voice sirf input/output wrapper hai upar se, aur internet sirf wahan touch karta hoon jahan local data se jawaab mil hi nahi sakta."